Embeddings Merge 101: A Practical Guide to Merging Embeddings

Have you identified several **fill-mask** (aka *language models* aka *embedding models*) which complement each other, and you want to use all of them to train, for example, a classifier on top of it? Easy-peasy.

The conditions you need to meet are:

- For a **simplified approach**, the embeddings must be of the same architecture (let's say, both *roberta-base* or both *bert-large-case*) and same dimensions.
- However, if you are very invested in merging models even from **different families (e.g, textual and image embeddings, or BERT and RoBERTa)**, take a look at the **complex approach**, which describes how you could even merge models from different families.

Simplified approach: Merging models of same architecture

Combining two fill-mask models into one in Hugging Face generally involves a few steps:

1. **Load the Models**: Load the two models you want to combine.
2. **Combine the Embeddings**: Merge the embeddings from both models.
3. **Create a New Model**: Integrate the combined embeddings into a new model.

Step 1: Load the Models

```
from transformers import AutoModelForMaskedLM, AutoTokenizer

model_name_1 = 'model_name_1'
model_name_2 = 'model_name_2'

# Load the models
model1 = AutoModelForMaskedLM.from_pretrained(model_name_1)
model2 = AutoModelForMaskedLM.from_pretrained(model_name_2)

# Load the tokenizers
tokenizer1 = AutoTokenizer.from_pretrained(model_name_1)
tokenizer2 = AutoTokenizer.from_pretrained(model_name_2)
```

Step 2: Combine the Embeddings

You need to ensure that both models have compatible tokenizers or you might need to handle token mappings. There are several techniques to do so:

Averaging Embeddings

The default technique is to sum the embeddings and divide by 2.

```
import torch

# Get embeddings from both models
embeddings1 = model1.base_model.embeddings.word_embeddings.weight
embeddings2 = model2.base_model.embeddings.word_embeddings.weight

# Ensure the dimensions match
assert embeddings1.shape == embeddings2.shape, "Embedding dimensions do not
match!"

# Combine embeddings (e.g., by averaging)
combined_embeddings = (embeddings1 + embeddings2) / 2
```

Concatenating the embeddings

Concatenating the embeddings from two models increases the dimensionality of the resulting embeddings, which may capture more information from both models.

```
import torch

# Concatenate embeddings
combined_embeddings = torch.cat((embeddings1, embeddings2), dim=-1)

# Adjust the embedding layer of the new model to match the new dimension
new_model.config.hidden_size = combined_embeddings.shape[-1]
new_model.base_model.embeddings.word_embeddings =
torch.nn.Embedding.from_pretrained(combined_embeddings)
```

Weighted Sum

Use a weighted sum to combine the embeddings. You can learn the weights during training or set them manually.

```
alpha = 0.7  # Weight for the first model
beta = 0.3   # Weight for the second model

# Combine embeddings with weights
combined_embeddings = alpha * embeddings1 + beta * embeddings2

# Set the combined embeddings to the new model
new_model.base_model.embeddings.word_embeddings.weight =
torch.nn.Parameter(combined_embeddings)
```

Linear Transformations

Use a linear transformation to combine the embeddings. This approach allows learning a transformation matrix during training.

```
import torch.nn as nn

class LinearCombiner(nn.Module):
    def __init__(self, embedding_dim):
        super(LinearCombiner, self).__init__()
        self.transform = nn.Linear(embedding_dim * 2, embedding_dim)

    def forward(self, emb1, emb2):
        combined = torch.cat((emb1, emb2), dim=-1)
        return self.transform(combined)

combiner = LinearCombiner(embeddings1.shape[-1])
combined_embeddings = combiner(embeddings1, embeddings2)

# Set the combined embeddings to the new model
new_model.base_model.embeddings.word_embeddings.weight =
torch.nn.Parameter(combined_embeddings)
```

Use a Neural Network instead to combine them

Use a small neural network to learn how to combine the embeddings.

```
class EmbeddingCombiner(nn.Module):
    def __init__(self, embedding_dim):
        super(EmbeddingCombiner, self).__init__()
        self.fc1 = nn.Linear(embedding_dim * 2, embedding_dim)
        self.fc2 = nn.Linear(embedding_dim, embedding_dim)
        self.relu = nn.ReLU()

    def forward(self, emb1, emb2):
        combined = torch.cat((emb1, emb2), dim=-1)
        combined = self.relu(self.fc1(combined))
        return self.fc2(combined)

combiner = EmbeddingCombiner(embeddings1.shape[-1])
combined_embeddings = combiner(embeddings1, embeddings2)

# Set the combined embeddings to the new model
new_model.base_model.embeddings.word_embeddings.weight =
torch.nn.Parameter(combined_embeddings)
```

Neural network + Attention Mechanism

Use an attention mechanism to learn how to combine embeddings. This method allows the model to weigh the importance of each embedding dynamically.

```
class AttentionCombiner(nn.Module):
    def __init__(self, embedding_dim):
        super(AttentionCombiner, self).__init__()
        self.attention = nn.MultiheadAttention(embed_dim=embedding_dim,
num_heads=1)

    def forward(self, emb1, emb2):
        combined = torch.stack((emb1, emb2), dim=0)
        attention_output, _ = self.attention(combined, combined, combined)
        return torch.mean(attention_output, dim=0)

combiner = AttentionCombiner(embeddings1.shape[-1])
combined_embeddings = combiner(embeddings1.unsqueeze(1),
embeddings2.unsqueeze(1)).squeeze(1)

# Set the combined embeddings to the new model
new_model.base_model.embeddings.word_embeddings.weight =
torch.nn.Parameter(combined_embeddings)
```

Step 3: Create a New Model

Create a new model with the combined embeddings. We can do this by initializing a new model and replacing its embeddings with the combined ones.

```
from transformers import BertConfig, BertForMaskedLM

# Use a configuration from one of the models or create a new one
config = BertConfig.from_pretrained(model_name_1)

# Create a new model
new_model = BertForMaskedLM(config)

# Replace the embeddings with the combined embeddings
new_model.base_model.embeddings.word_embeddings.weight =
torch.nn.Parameter(combined_embeddings)

# Save the new model
new_model.save_pretrained('combined_model')
```

Step 4: Save and Load the New Model

Save the model so you can load it later as needed.

```
# Save the new model and tokenizer
new_model.save_pretrained('path_to_combined_model')
tokenizer1.save_pretrained('path_to_combined_model')
```

Loading and Using the New Model

Now you can load and use the new model as usual.

```
from transformers import AutoModelForMaskedLM, AutoTokenizer

# Load the new model and tokenizer
new_model = AutoModelForMaskedLM.from_pretrained('path_to_combined_model')
tokenizer = AutoTokenizer.from_pretrained('path_to_combined_model')

# Example usage
input_text = "This is a [MASK] example."
inputs = tokenizer(input_text, return_tensors='pt')
outputs = new_model(**inputs)
```

Complex approach: What if the embeddings come from different families?

Combining embeddings from different model families, such as **textual** and **image embeddings** or a model`roberta-base` with `bert-base-uncased`, can be more challenging than combining embeddings from the same model family. This is because different model architectures may have different **embedding dimensions**, **tokenization strategies**, and even **pre-training objectives**. However, it's not impossible.

The steps now include include additionally:

1. **Tokenization Alignment:** Different models often have different tokenizers. To combine embeddings, you need to align the tokenization strategies. One approach is to use a unified tokenizer that works with both models, but this can be complex.
2. **Embedding Dimension Alignment:** If the embedding dimensions of the two models are different, you'll need to align them. This can be done using techniques like **linear transformation**, **zero-padding**, or **projection to a common space**.

Here's a more detailed example that combines embeddings from `roberta-base` and `bert-base-uncased`:

Step 1: Load the Models and Tokenizers

```
from transformers import AutoModelForMaskedLM, AutoTokenizer

# Load the models
model_name_1 = 'roberta-base'
model_name_2 = 'bert-base-uncased'

model1 = AutoModelForMaskedLM.from_pretrained(model_name_1)
model2 = AutoModelForMaskedLM.from_pretrained(model_name_2)

# Load the tokenizers
tokenizer1 = AutoTokenizer.from_pretrained(model_name_1)
tokenizer2 = AutoTokenizer.from_pretrained(model_name_2)
```

Step 2: Tokenization

You need to ensure the tokens from both tokenizers align. One way is to tokenize the input with both tokenizers and handle the alignment manually.

```
input_text = "This is a [MASK] example."

tokens1 = tokenizer1.tokenize(input_text)
tokens2 = tokenizer2.tokenize(input_text)

# Convert tokens to IDs
ids1 = tokenizer1.convert_tokens_to_ids(tokens1)
ids2 = tokenizer2.convert_tokens_to_ids(tokens2)

# Ensure alignment, e.g., by padding or truncating
max_length = max(len(ids1), len(ids2))
ids1 = ids1 + [tokenizer1.pad_token_id] * (max_length - len(ids1))
ids2 = ids2 + [tokenizer2.pad_token_id] * (max_length - len(ids2))
```

Step 3: Get Embeddings

Retrieve the embeddings from both models.

```
import torch

# Get embeddings
embeddings1 = model1.roberta.embeddings.word_embeddings.weight
embeddings2 = model2.bert.embeddings.word_embeddings.weight
```

Step 4: Align Embedding Dimensions

If the embedding dimensions differ, use a linear layer to project them to a common dimension.

```
import torch.nn as nn

# Assuming embeddings1 and embeddings2 have different dimensions
dim1 = embeddings1.size(1)
dim2 = embeddings2.size(1)
common_dim = max(dim1, dim2)

# Linear layers to project to common dimension
linear1 = nn.Linear(dim1, common_dim)
linear2 = nn.Linear(dim2, common_dim)

projected_embeddings1 = linear1(embeddings1)
projected_embeddings2 = linear2(embeddings2)
```

Step 5: Combine the Embeddings

Combine the projected embeddings using a chosen technique (e.g., concatenation, weighted sum).

```
# Combine embeddings, for example, by averaging
combined_embeddings = (projected_embeddings1 + projected_embeddings2) / 2

# Create a new embedding layer
new_embedding_layer = nn.Embedding.from_pretrained(combined_embeddings)
```

Step 6: Integrate into a New Model

Integrate the combined embeddings into a new model architecture.

```
from transformers import BertConfig, BertForMaskedLM

# Create a new configuration
config = BertConfig.from_pretrained(model_name_2)

# Initialize a new model
new_model = BertForMaskedLM(config)

# Replace the embeddings with the combined embeddings
new_model.bert.embeddings.word_embeddings = new_embedding_layer
```

```
# Save the new model
new_model.save_pretrained('path_to_combined_model')
tokenizer2.save_pretrained('path_to_combined_model')
```